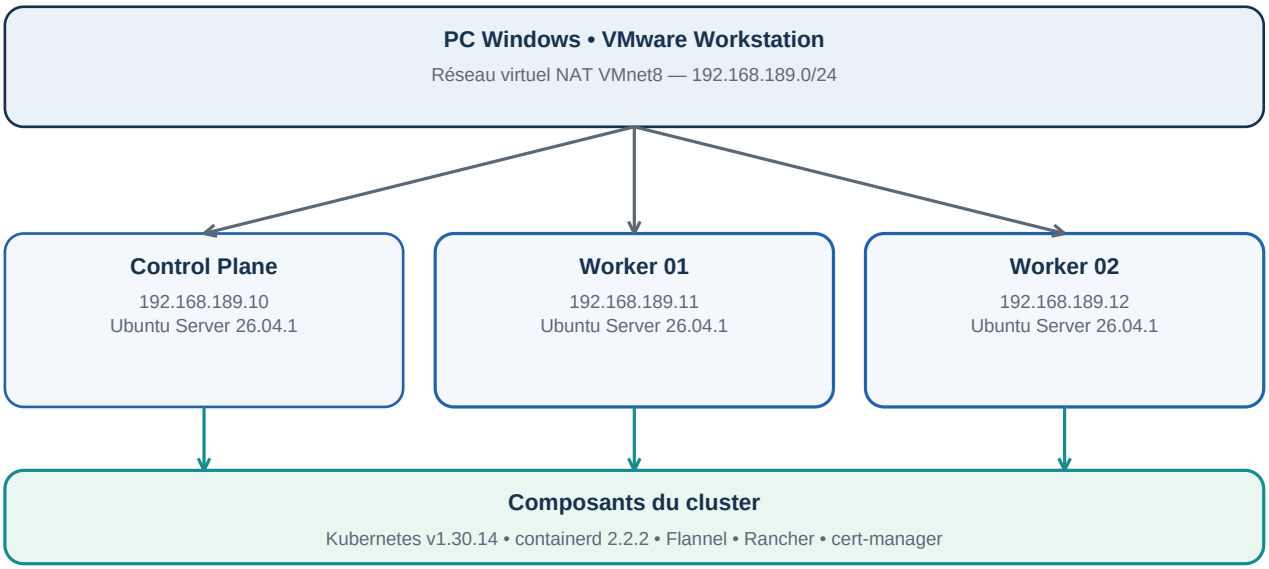
LABORATOIRE KUBERNETES

Déploiement, réseau, Services et gestion des applications
sur VMware Workstation et Ubuntu Server



Rapport technique — état au 3 novembre 2026

Document de capitalisation des manipulations réalisées, des commandes utilisées, des résultats observés et des concepts Kubernetes associés.

Environnement : Windows • VMware Workstation • Ubuntu Server 26.04.1 LTS • Kubernetes v1.30.14

Sommaire

1. Objectifs et périmètre du laboratoire
2. Infrastructure VMware et adressage réseau
3. Installation et état du cluster Kubernetes
4. Découverte des ressources et namespaces
5. Déploiement de Pods Nginx
6. Accès aux Pods avec kubectl port-forward
7. Labels, Service ClusterIP et endpoints
8. DNS interne et communication Pod-to-Service
9. Deployment, ReplicaSet et auto-réparation
10. Analyse de la répartition sur les Workers
11. Service du Deployment et validation HTTP
12. Bilan global, commandes de diagnostic et suite du laboratoire

Note de lecture : les adresses IP, noms de ressources et sorties reproduisent les éléments visibles dans les commandes partagées. Les schémas représentent l'architecture logique observée, pas une capture d'écran de Rancher.

1. Objectifs et périmètre du laboratoire

L'objectif est de construire un cluster Kubernetes local à trois machines virtuelles, puis de comprendre le cycle de vie d'une application conteneurisée : création d'un Pod, exposition par un Service, résolution DNS interne, orchestration avec un Deployment et remplacement automatique d'un Pod supprimé.

Le laboratoire permet également de vérifier la séparation des responsabilités entre les objets Kubernetes. Un Pod exécute le conteneur ; un ReplicaSet maintient un nombre de Pods ; un Deployment pilote les ReplicaSets et les mises à jour ; un Service fournit un point d'accès stable aux Pods sélectionnés.

Périmètre effectivement réalisé

Domaine	Éléments réalisés
Infrastructure	3 VM Ubuntu Server sur VMware Workstation, réseau NAT VMnet8.
Cluster	1 Control Plane et 2 Workers, tous Ready.
Réseau Kubernetes	Flannel, CoreDNS, kube-proxy.
Plateforme d'administration	Rancher, Fleet et cert-manager présents et Running.
Application	Pods Nginx, Service ClusterIP, Pod de test BusyBox, Deployment Nginx à 3 réplicas.
Validation	Endpoints, DNS interne, requête HTTP, remplacement automatique d'un Pod.

Ce document couvre les manipulations consignées jusqu'au test réussi du Service nginx-deployment-service le 3 novembre 2026 à 19:04 UTC. Les exercices de scaling, Rolling Update et Rollback proposés ensuite ne sont pas considérés comme déjà exécutés.

2. Infrastructure VMware et adressage réseau

Le cluster fonctionne dans VMware Workstation sur un réseau virtuel NAT (VMnet8). Le NAT permet aux machines virtuelles d'accéder au réseau externe en passant par le réseau virtuel VMware, tout en conservant un adressage privé entre les VM.

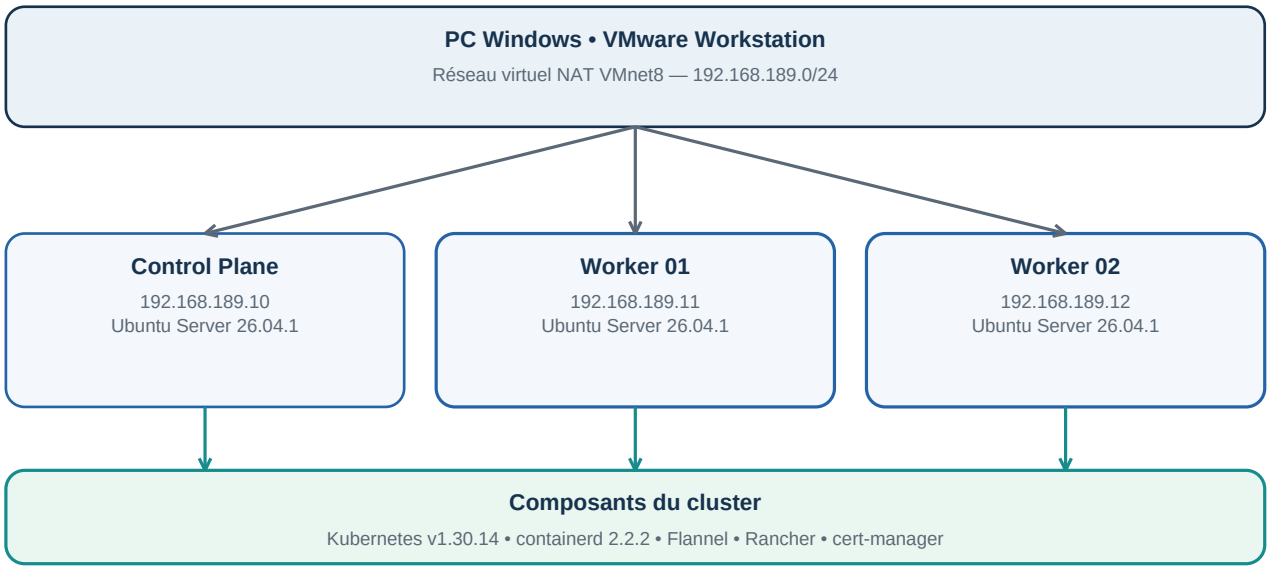


Figure 1 — Vue logique de l'infrastructure VMware et des trois machines virtuelles.

Rôle	Nom Kubernetes	Adresse IPv4	OS / runtime
Control Plane	k8s-control-plane	192.168.189.10	Ubuntu 26.04.1 / containerd 2.2.2
Worker 01	k8s-worker-01	192.168.189.11	Ubuntu 26.04.1 / containerd 2.2.2

Rôle	Nom Kubernetes	Adresse IPv4	OS / runtime
Worker 02	k8s-worker-02	192.168.189.12	Ubuntu 26.04.1 / containerd 2.2.2

Paramètres réseau consignés

Paramètre	Valeur
Réseau VMnet8	192.168.189.0/24
Passerelle NAT VMware	192.168.189.2
Adresse hôte Windows	192.168.189.1
Interface réseau Ubuntu	ens33
Adresses des nœuds	.10, .11 et .12

Le cluster utilise également le réseau de Pods 10.244.0.0/16. Les adresses observées sont réparties par nœud : 10.244.1.x sur Worker 01 et 10.244.2.x sur Worker 02. Ces adresses de Pods sont distinctes des adresses du réseau VMnet8.

Attention à la distinction : 192.168.189.x correspond au réseau des machines virtuelles ; 10.244.x.x correspond au réseau interne des Pods géré par le plugin CNI Flannel.

3. Installation et état du cluster Kubernetes

Le cluster est constitué d'un nœud de contrôle et de deux nœuds de travail. La version Kubernetes affichée sur les trois nœuds est v1.30.14. Le runtime de conteneurs est containerd 2.2.2.

Élément	État observé
k8s-control-plane	Ready — rôle control-plane
k8s-worker-01	Ready
k8s-worker-02	Ready
Version Kubernetes	v1.30.14 sur les 3 nœuds
Runtime	containerd://2.2.2
Système	Ubuntu 26.04.1 LTS

Commande de contrôle

```
kubectl get nodes -o wide
```

Composants d'infrastructure visibles dans les namespaces système

Composant	Namespace	Rôle
CoreDNS	kube-system	Résolution DNS des Services et des noms internes.
kube-proxy	kube-system	Mise en œuvre des règles réseau associées aux Services.
Flannel	kube-flannel	Réseau Pod-à-Pod du cluster.
Rancher	cattle-system	Interface d'administration Kubernetes.
Rancher Webhook	cattle-system	Admission et validation de ressources Rancher.
Fleet	cattle-fleet-system / cattle-fleet-local-system	Gestion de déploiements et de configurations.
cert-manager	cert-manager	Gestion du cycle de vie des certificats.

Les pods système essentiels étaient Running lors du relevé. Certains pods helm-operation ont été observés en état Completed, ce qui correspond à des tâches Helm terminées. Un ancien pod helm-operation était en Error dans une sortie intermédiaire, mais les composants Rancher et webhook étaient Running ; cet élément isolé ne suffit pas à conclure à une panne actuelle.

Commande de contrôle global

```
kubectl get pods -A
```

Le suffixe -A signifie --all-namespaces : il permet de voir les Pods de tous les namespaces, et pas uniquement ceux du namespace courant.

4. Découverte des ressources et namespaces

La commande kubectl get pods, sans option, interroge le namespace courant — ici default. Elle peut donc afficher « No resources found in default namespace » même si le cluster contient de nombreux Pods dans kube-system, cattle-system ou cert-manager.

Commande	Portée
kubectl get pods	Pods du namespace courant.
kubectl get pods -A	Pods de tous les namespaces.
kubectl get namespaces	Liste des namespaces.
kubectl get services	Services du namespace courant.
kubectl get deployments	Deployments du namespace courant.

Le namespace default est l'espace de travail utilisé pour les premiers essais Nginx. Les composants Kubernetes et Rancher sont installés dans leurs propres namespaces afin de séparer les ressources et leur administration.

Un namespace est un mécanisme de séparation logique à l'intérieur d'un même cluster. Il ne crée pas, à lui seul, une machine virtuelle ou un cluster distinct.

5. Déploiement de Pods Nginx

Deux Pods Nginx indépendants ont été créés à l'aide de manifests YAML : nginx-test sur Worker 02 et nginx-worker01 sur Worker 01. Ces Pods ont été créés directement comme ressources Pod, sans Deployment.

Pod	Nœud	Adresse IP Pod	État
nginx-test	k8s-worker-02	10.244.2.39	Running
nginx-worker01	k8s-worker-01	10.244.1.65	Running

Un premier essai avec kubectl run nginx-test --image=nginx --port=80 a affiché ContainerCreating et l'événement Pulling image. Le Pod a ensuite été supprimé. La création à partir du manifeste YAML a ensuite abouti et le Pod est devenu Running. La sortie initiale ne permet pas d'affirmer que le Pod était en panne : elle montre simplement que le téléchargement ou la création du conteneur était encore en cours au moment de l'observation.

Commandes utilisées

```
kubectl apply -f nginx-test.yaml
kubectl apply -f nginx-worker01.yaml
kubectl get pods -o wide
kubectl describe pod nginx-test
```

La commande describe est particulièrement utile pour diagnostiquer un Pod : elle expose son nœud d'affectation, son image, son état, ses volumes, ses conditions et les événements Kubernetes.

6. Accès aux Pods avec kubectl port-forward

Un transfert de port a été lancé vers le Pod nginx-test :

```
kubectl port-forward pod/nginx-test 8080:80
```

Cette commande crée un tunnel temporaire entre le port 8080 de la machine depuis laquelle kubectl est exécuté et le port 80 du Pod. Le terminal reste occupé tant que le transfert est actif.

Un premier essai de curl après Ctrl+C a renvoyé « Failed to connect », ce qui est normal : le tunnel avait été arrêté. Lors du second essai, la connexion a été traitée pendant que port-forward était actif.

Règle pratique

Laisser le terminal port-forward ouvert. Dans un second terminal, exécuter curl. Arrêter le tunnel avec Ctrl+C uniquement après le test.

7. Labels, Service ClusterIP et endpoints

Les deux Pods statiques ont reçu le label commun app=nginx :

```
kubectl label pod nginx-test app=nginx
kubectl label pod nginx-worker01 app=nginx
kubectl get pods --show-labels
```

Un Service nommé nginx-service a ensuite été créé. Il sélectionne les Pods qui portent le label app=nginx et expose leur port 80 derrière une adresse virtuelle stable.

```
apiVersion: v1
kind: Service
metadata:
name: nginx-service
spec:
type: ClusterIP
selector:
app: nginx
ports:
- name: http
protocol: TCP
port: 80
targetPort: 80
```

Propriété	Valeur observée
Service	nginx-service
Type	ClusterIP
ClusterIP	10.110.107.145
Port du Service	80/TCP
Endpoints	10.244.1.65:80 et 10.244.2.39:80

Le Service ne contient pas les Pods. Il les sélectionne dynamiquement grâce au selector et Kubernetes maintient la liste des endpoints correspondants. L'adresse ClusterIP reste le point d'accès stable, même si les adresses IP des Pods changent.

Commandes de vérification

```
kubectl get services
kubectl get endpoints nginx-service
kubectl describe service nginx-service
```

Le type ClusterIP rend le Service accessible depuis le réseau du cluster. Il ne publie pas automatiquement l'application sur le réseau local Windows ou sur Internet.

8. DNS interne et communication Pod-to-Service

Un Pod de diagnostic dns-test a été créé à partir de BusyBox et maintenu en fonctionnement pendant une heure :

```
kubectl run dns-test \
--image=busybox:1.36 \
--restart=Never \
--command -- sleep 3600
```

Le Pod a obtenu l'adresse 10.244.1.66 sur Worker 01. Une résolution DNS du nom court nginx-service a affiché plusieurs réponses NXDOMAIN pour des suffixes intermédiaires, puis a correctement résolu le nom complet nginx-service.default.svc.cluster.local vers 10.110.107.145.

La commande explicite suivante a fourni une réponse DNS propre :

```
kubectl exec dns-test -- nslookup nginx-service.default.svc.cluster.local
```

Pourquoi le nom complet ?

Dans Kubernetes, le nom DNS d'un Service suit généralement la forme : nom-service.namespace.svc.cluster.local. Le client BusyBox utilise une liste de suffixes DNS pour tenter de compléter un nom court. Les NXDOMAIN intermédiaires ne signifient donc pas que le Service est inaccessible lorsque le nom complet est correctement résolu.

Le test HTTP depuis le Pod BusyBox a ensuite réussi :

```
kubectl exec dns-test -- wget -qO- http://nginx-service
```

La page « Welcome to nginx! » a été renvoyée. Ce test valide une chaîne fonctionnelle : le Pod client interroge CoreDNS, obtient la ClusterIP du Service, puis accède au Service qui dirige le trafic vers un Pod Nginx.

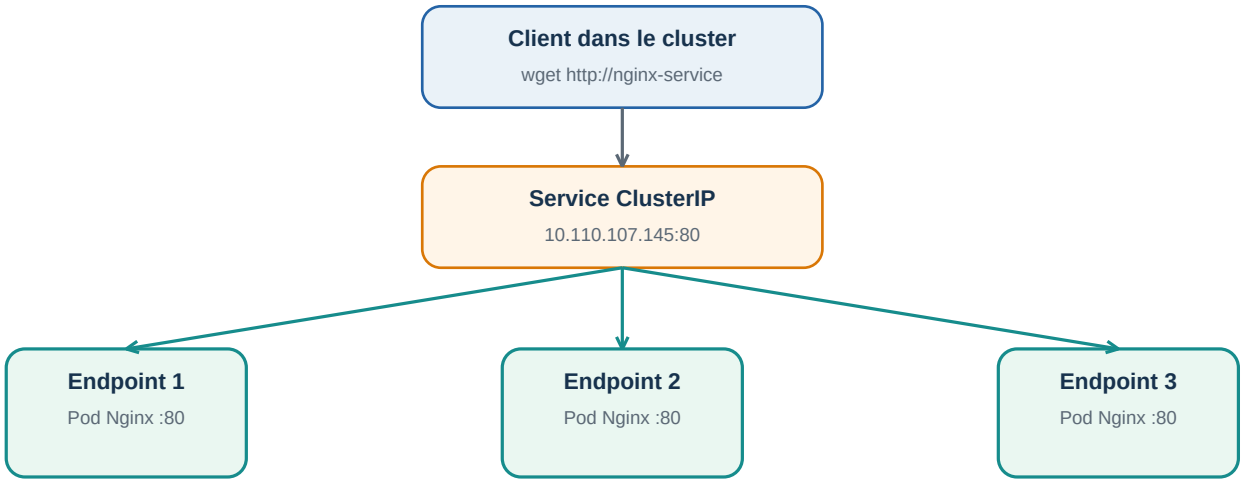


Figure 2 — Chemin logique de la requête HTTP depuis le Pod client vers les endpoints Nginx.

9. Deployment, ReplicaSet et auto-réparation

Après les Pods créés manuellement, un Deployment a été créé dans le namespace default à partir de nginx-deployment.yaml. Il demande trois réplicas de l'image nginx:latest.

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: nginx-deployment
spec:
replicas: 3
selector:
matchLabels:
app: nginx-deployment
template:
metadata:
labels:
app: nginx-deployment
spec:
containers:
- name: nginx
image: nginx:latest
imagePullPolicy: IfNotPresent
ports:
- containerPort: 80
```

Le Deployment a créé un ReplicaSet nommé nginx-deployment-75f6447756. Celui-ci a créé et maintenu trois Pods. Le résultat observé était READY 3/3, UP-TO-DATE 3 et AVAILABLE 3.

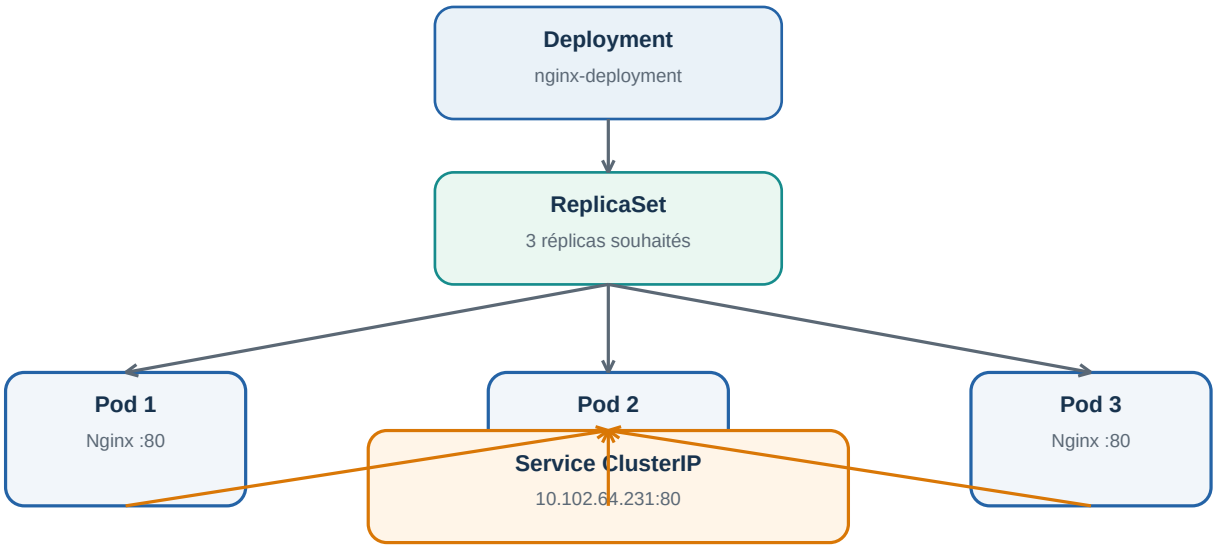


Figure 3 — Relation hiérarchique entre Deployment, ReplicaSet et Pods, avec le Service comme point d'accès aux Pods.

Test d'auto-réparation

Le Pod nginx-deployment-75f6447756-2tkk2 a été supprimé manuellement. Le ReplicaSet a détecté que le nombre de Pods était inférieur à la valeur souhaitée et a créé un nouveau Pod, nginx-deployment-75f6447756-vqp2z. Le nombre de réplicas est revenu à trois.

```
kubectl delete pod nginx-deployment-75f6447756-2tkk2
kubectl get pods -l app=nginx-deployment -w
```

C'est le contrôleur ReplicaSet qui maintient le nombre de Pods désiré. Le Deployment pilote le ReplicaSet et permet notamment de gérer les évolutions de la définition de l'application. Un Pod supprimé n'est pas « ressuscité » : un nouveau Pod est créé, avec un nouveau nom et généralement une nouvelle adresse IP.

À retenir : créer directement un Pod est utile pour les tests, mais pour une application qui doit rester disponible, il est généralement préférable de la gérer avec un Deployment.

10. Analyse de la répartition sur les Workers

Après le remplacement du Pod supprimé, la répartition relevée pour le Deployment était la suivante :

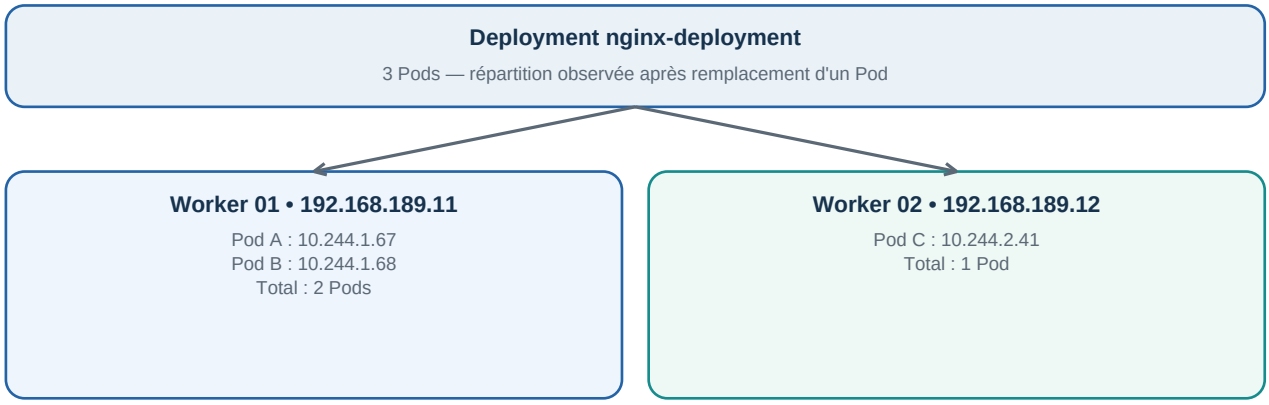


Figure 4 — Répartition observée des trois Pods du Deployment sur les deux Workers.

Nœud	Pods du Deployment	Nombre
k8s-worker-01 — 192.168.189.11	10.244.1.67 ; 10.244.1.68	2
k8s-worker-02 — 192.168.189.12	10.244.2.41	1
k8s-control-plane — 192.168.189.10	Aucun Pod du Deployment dans le relevé	0

Interprétation

Kubernetes a placé les Pods sur des nœuds disponibles selon les ressources et contraintes de planification. Avec trois réplicas et deux Workers, une répartition 2 + 1 est parfaitement possible. Kubernetes ne garantit pas automatiquement une répartition strictement égale entre les nœuds.

Le Control Plane n'a pas hébergé ces Pods dans le relevé. C'est cohérent avec un cluster kubeadm standard où le nœud de contrôle porte normalement une taint empêchant l'ordonnancement des applications ordinaires, sauf tolération ou modification de cette configuration.

Pour contrôler la répartition

```
kubectl get pods -l app=nginx-deployment -o wide
kubectl get nodes
```

Pour imposer ou favoriser une distribution entre les nœuds, les mécanismes à étudier sont topologySpreadConstraints et podAntiAffinity. Ils expriment une politique de placement. Il faut les configurer dans le template du Pod du Deployment ; ils ne doivent pas être confondus avec le nombre de réplicas.

11. Service du Deployment et validation HTTP

Un second Service, nginx-deployment-service, a été créé pour exposer exclusivement les Pods du Deployment. Il utilise le label app=nginx-deployment, différent du label app=nginx des Pods statiques.

```
apiVersion: v1
kind: Service
metadata:
name: nginx-deployment-service
spec:
type: ClusterIP
selector:
app: nginx-deployment
ports:
- name: http
```

```
protocol: TCP
port: 80
targetPort: 80
```

Propriété	Valeur observée
Service	nginx-deployment-service
Type	ClusterIP
ClusterIP	10.102.64.231
Port	80/TCP
Endpoints	10.244.1.67:80, 10.244.1.68:80, 10.244.2.41:80

La commande kubectl get endpoints nginx-deployment-service a confirmé trois endpoints. Le port-forward a ensuite été lancé :

```
kubectl port-forward svc/nginx-deployment-service 8081:80
```

Depuis un second terminal, la commande curl -I http://127.0.0.1:8081 a renvoyé HTTP/1.1 200 OK, avec Server: nginx/1.31.8. Le code HTTP 200 confirme que la requête a atteint un serveur Nginx et que celui-ci a répondu avec succès.

Résultat de validation



Figure 5 — Synthèse visuelle des principaux tests réussis.

Le port-forward reste un mécanisme temporaire de diagnostic. Il ne transforme pas le Service ClusterIP en service exposé de façon permanente sur le réseau local.

12. Bilan global et commandes de référence

Le laboratoire a validé les briques essentielles du premier déploiement d'une application sur Kubernetes : disponibilité des nœuds, exécution des conteneurs, réseau Pod, DNS interne, sélection des Pods par labels, accès via un Service, gestion des réplicas et auto-réparation.

Test	Commande ou preuve	Résultat
État du cluster	kubectl get nodes -o wide	3 nœuds Ready
Pods Nginx statiques	kubectl get pods -o wide	2 Pods Running sur 2 Workers
Service statique	kubectl get endpoints nginx-service	2 endpoints
DNS interne	nslookup nginx-service.default.svc.cluster.local	ClusterIP résolue
Communication HTTP interne	wget -qO- http://nginx-service	Page Nginx renvoyée
Deployment	kubectl get deployments	3/3 disponibles
Auto-réparation	Suppression d'un Pod du Deployment	Nouveau Pod créé
Service du Deployment	kubectl get endpoints nginx-deployment-service	3 endpoints
HTTP via port-forward	curl -I http://127.0.0.1:8081	HTTP 200 OK

Commandes essentielles à conserver

```
# Vue d'ensemble
kubectl get nodes -o wide
kubectl get namespaces
kubectl get pods -A

# Ressources applicatives du namespace default
kubectl get pods -o wide
kubectl get deployments
kubectl get replicasets
kubectl get services
kubectl get endpoints

# Diagnostic d'une ressource
kubectl describe pod nginx-test
kubectl describe deployment nginx-deployment
kubectl describe service nginx-deployment-service

# Logs et exécution dans un conteneur
kubectl logs <nom-du-pod>
kubectl exec -it <nom-du-pod> -- sh

# Filtrer les Pods par label
kubectl get pods -l app=nginx-deployment -o wide
```

Nettoyage des ressources de test (à effectuer uniquement lorsque les tests sont terminés)

```
# Supprimer les objets créés directement pour les tests
kubectl delete pod nginx-test nginx-worker01 dns-test
kubectl delete service nginx-service

# Supprimer le Deployment et son Service dédié
kubectl delete -f nginx-deployment-service.yaml
kubectl delete -f nginx-deployment.yaml
```

Ne pas exécuter ce nettoyage si tu souhaites conserver l'environnement actuel. La suppression du Deployment supprime également les Pods qu'il gère. Les composants Rancher, cert-manager, Flannel et Kubernetes ne sont pas concernés par ces commandes.

Prochaines étapes recommandées

Étape	Compétence à acquérir
1. Scaling	Modifier replicas et observer l'ajout/retrait de Pods.
2. Rolling Update	Changer la version de l'image et observer le déploiement progressif.
3. Rollback	Consulter l'historique et revenir à une révision précédente.
4. Probes	Configurer readinessProbe, livenessProbe et startupProbe.
5. ConfigMap et Secret	Externaliser la configuration et gérer les paramètres sensibles.
6. Volumes persistants	Comprendre PV, PVC et StorageClass.
7. Ingress	Publier des applications HTTP/HTTPS avec un contrôleur Ingress.
8. Scheduling	Étudier labels de nœuds, taints, tolerations, affinity et topology spread.

Conclusion

Le cluster local est opérationnel et les tests applicatifs consignés sont concluants. Le point de progression le plus important est désormais de passer des déploiements de test à une exploitation plus structurée : mises à jour contrôlées, vérifications de santé, configuration séparée de l'image, stockage et exposition HTTP. Ces étapes permettront de consolider les bases avant des déploiements plus proches d'un environnement professionnel.

Rapport établi à partir des sorties de terminal et des configurations communiquées dans la conversation. Les schémas sont des illustrations techniques originales de l'architecture et des flux décrits.